Module System

Previous: External Entity Definition Sources | Documentation Home | Next: Add Or Choose A Data Access Service

Coredeux modules are optional entity-level capabilities configured through YAML and executed by the framework strategy layer.

Modules allow the framework to add behavior without hardcoding that behavior into the service API or the data-access SPI.

Why Modules Exist

Modules let the framework compose behavior per entity.

Examples:

- one entity may use validators and hooks
- another entity may use validators, hooks, and audit
- another entity may use workflow, import/export, or another custom capability
- future agent-aware modules may expose selected entity operations as MCP capabilities while preserving validation, lifecycle hooks, and audit behavior

This makes the framework extensible without forcing every entity to use every capability.

Current Built-In Module Types

validators

Purpose:

- validate entity state before write operations

Contract:

- `CoredeuxEntityValidator`

Expected behavior:

- return a list of `ValidationError`
- the framework aggregates validation errors and raises a validation exception when needed

hooks

Purpose:

- respond to framework lifecycle phases

Contract:

- `CoredeuxEntityHook`

Hooks are lifecycle-oriented and are useful for enrichment, side effects, and entity-specific

framework behavior.

audit

Purpose:

- emit audit behavior based on business-level operations

Contract:

- `CoredeuxEntityAuditHandler`

Audit configuration uses operations instead of raw framework phases.

Example:

```
- name: audit
  enabled: true
  handlers:
    - defaultAuditHandler
  config:
    operations:
      - SAVE
      - UPDATE
```

The framework maps those operations internally to the correct execution points.

Module Resolution

Module execution works in three steps:

1. the framework resolves the entity definition
2. it finds enabled modules configured for that entity
3. it dispatches to the corresponding `CoredeuxEntityModuleHandler`

The handler then resolves the configured Spring beans and executes them.

Phase-Controlled Execution

The strategy layer keeps module dispatch generic. If a module is enabled for an entity and a matching `CoredeuxEntityModuleHandler` exists, the strategy calls that handler for the current phase.

The handler decides whether any real work should happen for that phase. This is how module behavior stays configurable without adding module-specific branches to `DefaultCoredeuxStrategy`.

The demo `workflows` module shows the pattern:

```
- name: workflows
  enabled: true
  handlers:
    - customerApprovalWorkflow
  config:
    phases:
      - after-save
```

The workflow module handler reads `config.phases`. If the current phase is not listed, it returns without resolving or executing the configured workflow handlers. If `config.phases` is omitted, the handler can choose to run for every phase, or it can enforce a stricter module-specific default.

Use this pattern when a custom module should be available across the framework but only execute at selected lifecycle points.

Module Configuration Shape

Each module entry can define:

- `name`
- `enabled`
- `handlers`
- `config`

Example:

```
- name: validators
  enabled: true
  handlers:
    - customerRequiredFieldsValidator
    - customerBusinessRuleValidator
```

External Module Execution

Coredeux also exposes `CoredeuxModuleService` so modules can be invoked outside the normal CRUD path.

Typical use cases:

- reprocessing existing entities
- import/export flows
- admin operations
- replaying framework behavior explicitly

The framework still derives request and lifecycle context internally when external module execution is used.

Guidance For New Module Types

When adding a new module type:

1. keep the top-level entity contract unchanged
2. model the feature under the entity's `modules` list
3. add a dedicated handler contract only if needed
4. add a `CoredeuxEntityModuleHandler` implementation in the relevant module

5. keep module-specific configuration inside the module's `config` block

This keeps `coredeux-core` stable while still allowing the framework to grow.

Previous: External Entity Definition Sources | Documentation Home | Next: Add Or Choose A Data Access Service